MBGA Delivery App — API Integration Specification

Audience: Backend Engineering Team **Purpose:** Exact contract for every endpoint the DeliveryApp frontend calls (or needs), derived directly from `src/api/`, `src/types/`, and `src/api/interceptors/errorInterceptor.ts`. Postman/Swagger-ready — every request/response below is the literal shape the frontend sends and expects.

> **Source of truth note:** This replaces the earlier stub in this file (which listed routes like `/users/me`, `/delivery/active`, `POST /orders` that were never implemented). Everything below is generated from the actual TypeScript service layer, not from the original design doc.

---

1. Base URL & Environment

```
EXPO_PUBLIC_API_BASE_URL=https://api.mbga.com   # placeholder — see open item #1
```

All routes below are **relative to `BASE_URL`**. The frontend does not currently prefix requests with a version segment (no `/v1`) — see Open Items #1 to confirm this with the backend team before build-out.

Timeout: `10000ms` default (`EXPO_PUBLIC_API_TIMEOUT`, configurable).

---

2. Common Request Headers

Header	Value	Applies to
`Content-Type`	`application/json`	All requests
`Accept`	`application/json`	All requests
`Authorization`	`Bearer <access_token>`	All endpoints **except** `POST /auth/login`, `POST /auth/verify-otp`, `POST /auth/refresh`

Important: `POST /auth/logout` **does require** the Bearer token — it is not treated as a public endpoint by the frontend.

On any `401` response (excluding the `/auth/*` endpoints themselves), the frontend automatically calls `POST /auth/refresh` once and retries the original request with the new token. If refresh also fails, the session is cleared client-side and the user is routed to login.

---

3. Common Response Envelopes

Success

```
{
  "success": true,
  "data": { }
}
```

`data` is the payload documented per-endpoint below. List endpoints either return a bare array (`data: [...]`) or a paginated object — noted per endpoint.

Paginated list shape

```
{
  "success": true,
  "data": {
    "items": [ ],
    "page": 1,
    "totalPages": 5,
    "totalItems": 47
  }
}
```

Error

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable message shown to the driver",
    "statusCode": 400
  }
}
```

This exact `{ success, error: { code, message, statusCode } }` shape is required — it is parsed directly into an `ApiError` by `src/api/interceptors/errorInterceptor.ts` (`errorInterceptor.ts:54-64`). Any deviation (missing `error.code`, different nesting, etc.) falls back to a generic `UNKNOWN_ERROR` on the client, which degrades the UX.

---

4. Endpoint Summary

#	Domain	Method	Route	Auth
1	Auth	POST	`/auth/login`	No
2	Auth	POST	`/auth/verify-otp`	No
3	Auth	POST	`/auth/refresh`	No
4	Auth	POST	`/auth/logout`	Yes
5	Deliveries	GET	`/deliveries/today`	Yes
6	Deliveries	GET	`/deliveries/{delive`	Yes
7	Orders	GET	`/orders`	Yes
8	Orders	GET	`/orders/{orderId}`	Yes
9	Orders	GET	`/orders/history`	Yes
10	Delivery Execution	POST	`/deliveries/{delive`	Yes

#	Domain	Method	Route	Auth
11	Delivery Execution	POST	`/deliveries/{deliveryId}/confirm`	Yes
12	Delivery Execution	POST	`/deliveries/{delive`	Yes `omer-otp`
13	Driver Profile	GET	`/driver/profile`	Yes
14	Driver Profile	PATCH	`/driver/status` **(pro**	Yes
15	Driver Profile	PATCH	`/driver/language` **(proposed)**	Yes
16	Notifications	GET	`/notifications`	Yes
17	Notifications	PATCH	`/notifications/{notificationId}/read`	Yes
18	Inventory	GET	`/driver/inventory` **(**	Yes
19	Payments	GET	`/payments/methods`	Yes
20	Payments	DELETE	`/payments/methods/{`	Yes
21	Payments	GET	`/payments/history`	Yes

Rows marked **(proposed)** have no frontend service/type wired yet — see each section for why, and treat their schemas as a starting proposal, not a locked contract.

---

5. Domain: Authentication

5.1 Login with Phone

Requests an OTP for a registered driver phone number.

- **Method & Route:** `POST /auth/login`
- **Auth Required:** No
- **Headers:** `Content-Type: application/json`, `Accept: application/json`

Request Body

Field	Type	Required	Description
`phone`	string	**REQUIRED**	Driver's registered phone number, e.g. `"+91 98765 43210"`

```
{ "phone": "+91 98765 43210" }
```

Success Response — 200 OK

```
{
  "success": true,
  "data": {
    "otpToken": "otp_tok_9f3e2b7a1c",
    "expiresInSeconds": 300
```

```
  }
}
```

`otpToken` is an opaque handle the client must pass back to `/auth/verify-otp` alongside the OTP the driver receives.

Error Responses

```
// 400 — phone missing / malformed
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "A valid phone number is required."
```

```
// 404 — phone not registered as a driver
{ "success": false, "error": { "code": "DRIVER_NOT_FOUND", "message": "No driver account found for this ph
```

5.2 Verify Duty OTP

Exchanges the OTP + `otpToken` for a session (access + refresh tokens) and the driver profile.

- **Method & Route:** `POST /auth/verify-otp`
- **Auth Required:** No
- **Headers:** `Content-Type: application/json`, `Accept: application/json`

Request Body

Field	Type	Required	Description
`phone`	string	**REQUIRED**	Must match the phone used in step 5.1
`otp`	string	**REQUIRED**	The OTP code entered by the driver
`otpToken`	string	**REQUIRED**	The `otpToken` returned by `/auth/login`

```
{
  "phone": "+91 98765 43210",
  "otp": "482913",
  "otpToken": "otp_tok_9f3e2b7a1c"
}
```

Success Response — 200 OK

```
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.access...",
    "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.refresh...",
    "driver": {
      "id": "driver-1",
      "name": "Ravi Kumar",
      "phone": "+91 98765 43210",
      "vehicleNumber": "UP14 AB 1234",
      "role": "Driver",
      "onDuty": true,
      "avatarInitials": "RK"
```

```
    }
  }
}
```

`avatarInitials` on `driver` is optional — omit the field entirely rather than sending `null` if not applicable.

Error Responses

```
// 400 — OTP wrong format
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "Enter the 6-digit code.", "statusC
```

```
// 401 — OTP incorrect or otpToken expired
{ "success": false, "error": { "code": "OTP_INVALID", "message": "Incorrect or expired code. Request a ne
```

```
// 404 — driver record no longer exists
{ "success": false, "error": { "code": "DRIVER_NOT_FOUND", "message": "No driver account found for this p
```

5.3 Refresh Token

Silently exchanges a refresh token for a new access/refresh pair. Called automatically by the client on any `401`; not user-initiated.

- **Method & Route:** `POST /auth/refresh`
- **Auth Required:** No (the refresh token itself is the credential — sent in the body, not as a Bearer header)
- **Headers:** `Content-Type: application/json`, `Accept: application/json`

Request Body

Field	Type	Required	Description
`refreshToken`	string	**REQUIRED**	The refresh token issued at login

```
{ "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.refresh..." }
```

Success Response — 200 OK

```
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.newaccess...",
    "refreshToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.newrefresh..."
  }
}
```

Error Responses

```
// 401 — refresh token invalid, revoked, or expired
{ "success": false, "error": { "code": "TOKEN_EXPIRED", "message": "Session expired. Please log in again.
```

On this error the client wipes local session storage and forces re-login — it does **not** retry again.

5.4 Logout

Invalidates the current session server-side.

- **Method & Route:** `POST /auth/logout`
- **Auth Required: Yes** (Bearer)
- **Headers:** `Content-Type: application/json`, `Accept: application/json`, `Authorization: Bearer <access_token>`
- **Request Body:** None

Success Response — 200 OK

```
{ "success": true, "data": null }
```

The client ignores the response body entirely — any `2xx` is treated as success. An empty `200` or `204` is fine; please keep the standard envelope for consistency.

Error Responses

```
// 401 — already-invalid/expired token
{ "success": false, "error": { "code": "UNAUTHORIZED", "message": "Session already expired.", "statusCode
```

---

6. Domain: Deliveries & Orders

6.1 Today's Assigned Deliveries

Returns every delivery assigned to the logged-in driver for the current day.

- **Method & Route:** `GET /deliveries/today`
- **Auth Required:** Yes (Bearer)
- **Query Params:** None
- **Request Body:** None

Success Response — 200 OK

Returns a **bare array** in `data` (not the paginated wrapper) — the app expects the full day's list in one call.

```
{
  "success": true,
  "data": [
    {
      "id": "ORD1256",
      "orderNumber": "ORD1256",
      "customerName": "ABC Restaurant",
      "customerPhone": "+91 98765 43210",
      "address": "Sector 61, Noida",
      "timeSlotStart": "10:50 AM",
      "timeSlotEnd": "12:00 PM",
      "status": "pending",
      "distanceKm": 1.8,
      "items": [
        { "id": "ORD1256-cylinder", "kind": "cylinderDelivery", "label": "19 KG Cylinder", "quantity": 5 }
        { "id": "ORD1256-empty", "kind": "emptyCollection", "label": "Empty Cylinders to Collect", "quanti
      ]
    }
```

```
  ]
}
```

Field notes

Field	Type	Notes
`status`	enum	`"pending"` | `"in_progress"` | `"completed"`
`items[].kind`	enum	`"cylinderDelivery"` (going out to customer) | `"emptyCollection"` (coming back from customer)
`distanceKm`	number	Optional — omit if not computable
`completedAt`	string	Optional — only present once `status` is `"completed"` (e.g. `"11:20 AM"`)

Error Responses

```
{ "success": false, "error": { "code": "UNAUTHORIZED", "message": "Session expired. Please log in again."
```

6.2 Order Details by ID

Two routes return the **identical schema** (single `Order` / `Delivery` object) — the frontend calls the `orders` route from the Order List screen and the `deliveries` route from the active-delivery flow. Please implement both against the same underlying resource.

- **Method & Route:** `GET /orders/{orderId}`
- **Method & Route (alias, active-delivery context):** `GET /deliveries/{deliveryId}`
- **Auth Required:** Yes (Bearer)
- **Path Param:** `orderId` / `deliveryId` — string, **REQUIRED**

Success Response — 200 OK

```
{
  "success": true,
  "data": {
    "id": "ORD1259",
    "orderNumber": "ORD1259",
    "customerName": "City Bites",
    "customerPhone": "+91 98765 43213",
    "address": "Sector 50, Noida",
    "timeSlotStart": "09:00 AM",
    "timeSlotEnd": "10:00 AM",
    "status": "in_progress",
    "distanceKm": 0.9,
    "items": [
      { "id": "ORD1259-cylinder", "kind": "cylinderDelivery", "label": "19 KG Cylinder", "quantity": 3 },
      { "id": "ORD1259-empty", "kind": "emptyCollection", "label": "Empty Cylinders to Collect", "quantity
    ]
  }
}
```

Error Responses

```
// 404 — no order/delivery with this ID for this driver
{ "success": false, "error": { "code": "NOT_FOUND", "message": "Order not found.", "statusCode": 404 } }
```

```
{ "success": false, "error": { "code": "UNAUTHORIZED", "message": "Session expired. Please log in again."
```

6.3 Order List

- **Method & Route:** `GET /orders`
- **Auth Required:** Yes (Bearer)

Query Parameters

Param	Type	Required	Description
`page`	number	OPTIONAL	1-indexed page number
`limit`	number	OPTIONAL	Page size
`status`	enum	OPTIONAL	`"pending"` | `"in_progress"` | `"completed"`

```
GET /orders?page=1&limit=20&status=pending
```

Success Response — 200 OK

```
{
  "success": true,
  "data": {
    "items": [ /* Order[] — same shape as §6.2 */ ],
    "page": 1,
    "totalPages": 1,
    "totalItems": 12
  }
}
```

Error Responses

```
// 400 — invalid status/page/limit value
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "status must be one of pending, in_
```

6.4 Order History (with filters)

- **Method & Route:** `GET /orders/history`
- **Auth Required:** Yes (Bearer)

Query Parameters

Param	Type	Required	Description
`page`	number	OPTIONAL	1-indexed page number
`limit`	number	OPTIONAL	Page size
`status`	enum	OPTIONAL	`"pending"` | `"in_progress"` | `"completed"`
`period`	enum	OPTIONAL — **unconfirmed, see Open Items #2**	`"today"` | `"week"` | `"all"` — backs the Today/This Week/All tabs on the Delivery History screen

```
GET /orders/history?period=week&page=1&limit=20
```

Success Response — 200 OK


```
{
 "success": true,
 "data": {
   "items": [
     {
       "id": "ORD1254",
       "orderNumber": "ORD1254",
       "customerName": "The Grand Hotel",
       "customerPhone": "+91 98765 43221",
       "address": "Sector 50, Noida",
       "timeSlotStart": "10:00 AM",
       "timeSlotEnd": "11:00 AM",
       "status": "completed",
       "distanceKm": 1.2,
       "completedAt": "11:20 AM",
       "items": [
         { "id": "ORD1254-cylinder", "kind": "cylinderDelivery", "label": "19 KG Cylinder", "quantity": 7
         { "id": "ORD1254-empty", "kind": "emptyCollection", "label": "Empty Cylinders to Collect", "quan
       ]
     }
   ],
   "page": 1,
   "totalPages": 1,
   "totalItems": 4
 }
}
```


Error Responses


```
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "period must be one of today, week,
```


---

7. Domain: Delivery Execution Flow

Sequence per delivery: **start -> confirm -> verify-customer-otp**.

7.1 Start Delivery (Driver Geo-coordinates)

Called when the driver taps "Start Delivery" — sends current GPS location so the backend can confirm geofence proximity to the drop address.

- **Method & Route:** `POST /deliveries/{deliveryId}/start`
- **Auth Required:** Yes (Bearer)
- **Path Param:** `deliveryId` — string, **REQUIRED**

Request Body

Field	Type	Required	Description
`latitude`	number	**REQUIRED**	Driver's current latitude
`longitude`	number	**REQUIRED**	Driver's current longitude

```
{ "latitude": 28.5710, "longitude": 77.3620 }
```

Success Response — 200 OK

```
{
  "success": true,
  "data": {
    "isAtLocation": true,
    "distanceMetersFromDestination": 42
  }
}
```

`isAtLocation` drives whether the client lets the driver proceed straight to Confirm, or shows a "you're X meters away" warning.

Error Responses

```
// 400 — missing/invalid coordinates
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "latitude and longitude are require
```

```
// 404 — deliveryId doesn't exist or isn't assigned to this driver
{ "success": false, "error": { "code": "NOT_FOUND", "message": "Delivery not found.", "statusCode": 404 }
```

7.2 Confirm Quantities / Empty Cylinders Collected

- **Method & Route:** `POST /deliveries/{deliveryId}/confirm`
- **Auth Required:** Yes (Bearer)
- **Path Param:** `deliveryId` — string, **REQUIRED**

Request Body

Field	Type	Required	Description
`deliveredQuantity`	number	**REQUIRED**	Number of full cylinders actually delivered

Field	Type	Required	Description
`emptyCollectedQuantity`	number	**REQUIRED**	Number of empty cylinders collected back
`notes`	string	OPTIONAL	Free-text driver notes (e.g. partial delivery reason)

```
{
 "deliveredQuantity": 5,
 "emptyCollectedQuantity": 4,
 "notes": "Customer had only 4 empties on site; will return the 5th next visit."
}
```

Success Response — 200 OK

```
{
 "success": true,
 "data": {
   "deliveryId": "ORD1256",
   "customerOtpRequired": true
 }
}
```

If `customerOtpRequired` is `true`, the client proceeds to §7.3. If `false`, the delivery is considered complete at this step.

Error Responses

```
// 400 — quantities negative, non-numeric, or exceed the order's line items
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "deliveredQuantity cannot exceed th
```

```
{ "success": false, "error": { "code": "NOT_FOUND", "message": "Delivery not found.", "statusCode": 404 }
```

7.3 Customer Delivery Verification OTP

- **Method & Route:** `POST /deliveries/{deliveryId}/verify-customer-otp`
- **Auth Required:** Yes (Bearer)
- **Path Param:** `deliveryId` — string, **REQUIRED**

Request Body

Field	Type	Required	Description
`otp`	string	**REQUIRED**	OTP read out by the customer to confirm receipt

```
{ "otp": "738104" }
```

Success Response — 200 OK

```
{
  "success": true,
  "data": {
    "deliveryId": "ORD1256",
    "orderNumber": "ORD1256",
    "deliveredQuantity": 5,
    "emptyCollectedQuantity": 4,
    "completedAt": "2026-09-17T11:42:00.000Z"
  }
}
```

Error Responses

```
// 401 — wrong OTP
{ "success": false, "error": { "code": "OTP_INVALID", "message": "Incorrect code. Ask the customer to con
```

```
{ "success": false, "error": { "code": "NOT_FOUND", "message": "Delivery not found.", "statusCode": 404 }
```

---

8. Domain: Driver Profile & Status

8.1 Get Profile

- **Method & Route:** `GET /driver/profile`
- **Auth Required:** Yes (Bearer)
- **Request Body:** None

Success Response — 200 OK

```
{
  "success": true,
  "data": {
    "id": "driver-1",
    "name": "Ravi Kumar",
    "phone": "+91 98765 43210",
    "vehicleNumber": "UP14 AB 1234",
    "role": "Driver",
    "onDuty": true,
    "avatarInitials": "RK"
  }
}
```

Error Responses

```
{ "success": false, "error": { "code": "UNAUTHORIZED", "message": "Session expired. Please log in again."
```

8.2 Update On-Duty Status — [PROPOSED] PROPOSED, NOT YET WIRED

> **Flag for backend:** The `Driver` type already has an `onDuty: boolean` field (returned by login and profile), but **no frontend endpoint constant, service method, or screen currently calls an update route.** The spec below is a proposal based on the existing `Driver` shape — please confirm before building, and the frontend team will wire `userEndpoints.ts` / `userService.ts` to match once agreed.

- **Method & Route (proposed):** `PATCH /driver/status`
- **Auth Required:** Yes (Bearer)

Request Body

Field	Type	Required	Description
`onDuty`	boolean	**REQUIRED**	`true` = driver goes on duty, `false` = off duty

```
{ "onDuty": false }
```

Success Response — 200 OK (proposed)

```
{
  "success": true,
  "data": {
    "id": "driver-1",
    "name": "Ravi Kumar",
    "phone": "+91 98765 43210",
    "vehicleNumber": "UP14 AB 1234",
    "role": "Driver",
    "onDuty": false,
    "avatarInitials": "RK"
  }
}
```

Error Responses (proposed)

```
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "onDuty must be a boolean.", "statu
```

```
// e.g. driver has an in-progress delivery and cannot go off duty
{ "success": false, "error": { "code": "CONFLICT", "message": "Cannot go off duty with an active delivery
```

8.3 Change Language — [PROPOSED] PROPOSED, NOT YET WIRED

> **Flag for backend:** There is currently **no language/locale field anywhere in `src/types/`** and no i18n implementation in the app. If localization is planned, the frontend team needs to add a `languageCode` field to `Driver` first. The schema below is a placeholder for planning purposes only — do not treat it as committed.

- **Method & Route (proposed):** `PATCH /driver/language`
- **Auth Required:** Yes (Bearer)

Request Body (proposed)

Field	Type	Required	Description
`languageCode`	string (enum)	**REQUIRED**	ISO 639-1 code, e.g. `"en"`, `"hi"`

```
{ "languageCode": "hi" }
```

Success Response — 200 OK (proposed)

```
{
 "success": true,
 "data": { "languageCode": "hi" }
}
```

Error Responses (proposed)

```
{ "success": false, "error": { "code": "VALIDATION_ERROR", "message": "languageCode is not supported.", "
```

---

9. Domain: Notifications

9.1 Get Notifications List

- **Method & Route:** `GET /notifications`
- **Auth Required:** Yes (Bearer)
- **Request Body:** None

Success Response — 200 OK

Bare array in `data`, most recent first.

```
{
 "success": true,
 "data": [
   {
     "id": "notif-1",
     "type": "new_delivery",
     "title": "New Delivery Assigned",
     "subtitle": "#ORD1260 - Metro Hotel",
     "createdAt": "2026-09-17T10:38:00.000Z",
     "isRead": false
   },
   {
     "id": "notif-2",
     "type": "delivery_confirmed",
     "title": "Delivery Confirmed",
     "subtitle": "#ORD1254",
     "createdAt": "2026-09-17T09:40:00.000Z",
     "isRead": true
   }
 ]
}
```

`type` is a closed enum: `"new_delivery"` | `"delivery_confirmed"` | `"order_updated"` | `"system"`.

Error Responses

```
{ "success": false, "error": { "code": "UNAUTHORIZED", "message": "Session expired. Please log in again."
```

9.2 Mark Notification as Read

- **Method & Route:** `PATCH /notifications/{notificationId}/read`

- **Auth Required:** Yes (Bearer)
- **Path Param:** `notificationId` — string, **REQUIRED**
- **Request Body:** None

Success Response — 200 OK

```
{ "success": true, "data": null }
```

The client ignores the body — any `2xx` is treated as success.

Error Responses

```
{ "success": false, "error": { "code": "NOT_FOUND", "message": "Notification not found.", "statusCode": 4
```

---

10. Domain: Inventory / Vehicle Stock — [PROPOSED] PROPOSED, NOT YET WIRED

> **Flag for backend:** There is **no inventory or vehicle-stock type, endpoint, or screen in the current codebase at all.** The only related signal is `OrderItem.kind` (`"cylinderDelivery"` vs `"emptyCollection"`) on each order — implying truck stock should deplete/accrue as deliveries are confirmed (§7.2), but nothing currently surfaces truck-level totals back to the driver. The schema below is a reasonable starting proposal based on the domain (LPG cylinder delivery) — please review with product before backend build-out, and the frontend team will add `driver.types.ts` fields, `userEndpoints.ts` / a new `inventoryService.ts` to match.

- **Method & Route (proposed):** `GET /driver/inventory`
- **Auth Required:** Yes (Bearer)
- **Request Body:** None

Success Response — 200 OK (proposed)

```
{
  "success": true,
  "data": {
    "fullCylinderCount": 18,
    "emptyCylinderCount": 6,
    "vehicleCapacity": 30,
    "lastUpdatedAt": "2026-09-17T08:00:00.000Z"
  }
}
```

Field	Type	Description
`fullCylinderCount`	number	Full cylinders currently loaded on the truck, available to deliver
`emptyCylinderCount`	number	Empty cylinders collected so far today, awaiting depot drop-off
`vehicleCapacity`	number	Total cylinder capacity of the assigned vehicle

Field	Type	Description
`lastUpdatedAt`	string (ISO 8601)	Server timestamp of last stock recalculation

Error Responses (proposed)

```
{ "success": false, "error": { "code": "NOT_FOUND", "message": "No vehicle assigned to this driver.", "st
```

---

11. Domain: Payments (Scaffolded — confirm scope before building)

> **Note:** `src/screens/Payments/` exists and these endpoints are wired in `paymentService.ts`, but per the code's own `TODO` comment this domain was **not part of the original 13-screen spec** — confirm with product whether it ships in v1 before investing backend effort here.

11.1 List Payment Methods

- **Method & Route:** `GET /payments/methods`
- **Auth Required:** Yes (Bearer)

Success Response — 200 OK

```
{
 "success": true,
 "data": [
   { "id": "pm-1", "type": "upi", "label": "UPI - ravi@okhdfcbank", "isDefault": true },
   { "id": "pm-2", "type": "cash", "label": "Cash on Delivery", "isDefault": false }
 ]
}
```

`type` is currently an unconstrained string in the frontend type — recommend backend and frontend agree a closed enum (e.g. `"upi" | "card" | "cash" | "wallet"`).

11.2 Remove Payment Method

- **Method & Route:** `DELETE /payments/methods/{methodId}`
- **Auth Required:** Yes (Bearer)
- **Path Param:** `methodId` — string, **REQUIRED**

Success Response — 200 OK

```
{ "success": true, "data": null }
```

Error Responses

```
{ "success": false, "error": { "code": "NOT_FOUND", "message": "Payment method not found.", "statusCode":
```

11.3 Payment History

- **Method & Route:** `GET /payments/history`
- **Auth Required:** Yes (Bearer)

Query Parameters

Param	Type	Required	Description
`page`	number	OPTIONAL	1-indexed page number
`limit`	number	OPTIONAL	Page size

Success Response — 200 OK

```
{
 "success": true,
 "data": {
   "items": [
     { "id": "txn-1", "amount": 4250, "status": "completed", "createdAt": "2026-09-16T14:20:00.000Z" }
   ],
   "page": 1,
   "totalPages": 1,
   "totalItems": 1
 }
}
```

`status` is an unconstrained string in the frontend type — recommend agreeing a closed enum (e.g. `"pending" | "completed" | "failed"`).

---

12. Error Code Reference (Suggested Convention)

No fixed error-code enum exists in the frontend today — `ApiError.code` is passed through verbatim from whatever the backend sends. The codes used throughout this document are a **suggested convention**; please finalize the exact list with the frontend team so copy/handling (e.g. distinguishing `OTP_INVALID` from `OTP_EXPIRED` for retry UX) can be wired precisely.

Code	Typical `statusCode`	Meaning
`VALIDATION_ERROR`	400	Request body/query failed validation
`OTP_INVALID`	401	OTP or otpToken incorrect/expired
`UNAUTHORIZED`	401	Missing/invalid access token
`TOKEN_EXPIRED`	401	Refresh token invalid or expired
`FORBIDDEN`	403	Authenticated but not permitted (e.g. delivery not assigned to this driver)
`NOT_FOUND`	404	Resource does not exist
`DRIVER_NOT_FOUND`	404	Phone number not registered
`CONFLICT`	409	State conflict (e.g. can't go off-duty mid-delivery)
`RATE_LIMITED`	429	Too many requests (e.g. OTP resend spam)

Code	Typical `statusCode`	Meaning
`INTERNAL_ERROR`	500	Unhandled server error

Every error, regardless of code, **must** use the envelope in §3 — the frontend's error interceptor has no fallback parsing for any other shape.

---

13. Postman / Swagger Notes

- Suggested collection folder structure mirrors the section numbering above: `Auth`, `Deliveries & Orders`, `Delivery Execution`, `Driver Profile`, `Notifications`, `Inventory (proposed)`, `Payments`.
- Suggested environment variables: `{{baseUrl}}`, `{{accessToken}}`, `{{refreshToken}}`, `{{deliveryId}}`, `{{orderId}}`.
- All authenticated requests: set `Authorization: Bearer {{accessToken}}` at the collection level so individual requests don't need to repeat it.
- A Postman pre-request script that auto-refreshes `{{accessToken}}` via §5.3 on a stored 401 would mirror the app's own retry behavior and is a good smoke test for that flow specifically.

---

14. Open Items / Confirmations Needed from Backend

1. **Base URL / version prefix** — `apiConfig.ts` currently has an explicit `TODO: confirm endpoint base path (e.g. trailing /v1)`. None of the routes above assume a version segment; confirm whether `BASE_URL` will include one.
2. **`period` filter on Order History** (§6.4) — marked `TODO` in `order.types.ts` in the frontend code; confirm exact param name and accepted values (`today` / `week` / `all` assumed here).
3. **On-Duty Status update** (§8.2) — needs an endpoint; currently only readable via profile, not writable.
4. **Change Language** (§8.3) — needs both a backend field and a frontend `languageCode` addition to `Driver`; not started on either side.
5. **Inventory / Vehicle Stock** (§10) — entirely new domain; needs product sign-off on the exact fields before backend build-out.
6. **Payments domain scope** (§11) — confirm whether this ships in v1 at all before backend investment.
7. **Error code enum** (§12) — finalize the canonical list with the frontend team so client-side UX branching (retry copy, disabled states, etc.) can key off exact codes.